Experiment 'p_4_5_analyse_floats_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_5_analyse_floats_array
Natural language statement: Write a method that takes a sequence of floatingpoint numbers and returns: - the average of the values. - the smallest of the values. - the largest of the values. - the range, that is the difference between the smallest and largest.
Method signature: p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real, max: real, range: real)

Ensures

- `average == (sum(values[..]) / values.Length as real)`
- `forall v :: v in values[..] ==> min <= v`
- `forall v :: v in values[..] ==> max >= v`
- `range == max - min`
- `exists v :: v in values[..] && min == v`
- `exists v :: v in values[..] && max == v`

Requires

- `0 < values.Length`

Functional Code Given

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of floatingpoint numbers and returns: - the average
    ↪ of the values. - the smallest of the values. - the largest of the values. - the
    ↪ range, that is the difference between the smallest and largest.

The signature should be:

p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real, max:
    ↪ real, range: real)

The method should respect the following contract:

requires 0 < values.Length, ensures average == (sum(values[..]) / values.Length as real),
    ↪  ensures forall v :: v in values[..] ==> min <= v, ensures forall v :: v in values
    ↪ [..] ==> max >= v, ensures range == max - min, ensures exists v :: v in values[..]
    ↪  && min == v, ensures exists v :: v in values[..] && max == v

The contract uses the following dafny code:

function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfbpGRC3dbqmkIgMtUyDQluGieKN
Model: gpt-5.5-2026-04-23
Created at: 1780492061
Finish reason: stop
Usage: 638 tokens in, and 4461 tokens out

```
{
  var i := values.Length - 1;
  var total := values[i];
  min := values[i];
  max := values[i];
```

```
assert values[i..] == [values[i]];
assert sum(values[i..]) == values[i];
assert forall v :: v in values[i..] ==> min <= v;
assert forall v :: v in values[i..] ==> max >= v;
assert min in values[i..];
assert max in values[i..];

while 0 < i
  invariant 0 <= i < values.Length
  invariant total == sum(values[i..])
  invariant forall v :: v in values[i..] ==> min <= v
  invariant forall v :: v in values[i..] ==> max >= v
  invariant min in values[i..]
  invariant max in values[i..]
  decreases i
{
  var oldI := i;
  var oldMin := min;
  var oldMax := max;

  i := i - 1;
  var x := values[i];

  assert oldI == i + 1;
  assert values[i..] == [x] + values[oldI..];
  assert values[i..][0] == x;
  assert values[i..][1..] == values[oldI..];

  total := x + total;
  assert total == x + sum(values[oldI..]);
  assert sum(values[i..]) == x + sum(values[i..][1..]);
  assert sum(values[i..]) == total;

  if x < oldMin {
    min := x;
    assert min in values[i..];
    forall v | v in values[i..]
      ensures min <= v
    {
      if v == x {
      } else {
        assert v in values[oldI..];
        assert oldMin <= v;
        assert min < oldMin;
      }
    }
  } else {
    min := oldMin;
    assert min <= x;
    assert min in values[oldI..];
```

```
    assert min in values[i..];
    forall v | v in values[i..]
      ensures min <= v
    {
      if v == x {
      } else {
        assert v in values[oldI..];
        assert oldMin <= v;
      }
    }
  }

  if x > oldMax {
    max := x;
    assert max in values[i..];
    forall v | v in values[i..]
      ensures max >= v
    {
      if v == x {
      } else {
        assert v in values[oldI..];
        assert oldMax >= v;
        assert max > oldMax;
      }
    }
  } else {
    max := oldMax;
    assert max >= x;
    assert max in values[oldI..];
    assert max in values[i..];
    forall v | v in values[i..]
      ensures max >= v
    {
      if v == x {
      } else {
        assert v in values[oldI..];
        assert oldMax >= v;
      }
    }
  }
 }

 assert i == 0;
 assert values[i..] == values[..];
 assert total == sum(values[..]);
 assert min in values[..];
 assert max in values[..];

 average := total / (values.Length as real);
 range := max - min;
}
```

Final Program

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}

method p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real,
    ↪  max: real, range: real)
       requires 0 < values.Length
       ensures average == (sum(values[..]) / values.Length as real)
       ensures forall v :: v in values[..] ==> min <= v
       ensures forall v :: v in values[..] ==> max >= v
       ensures range == max - min
       ensures exists v :: v in values[..] && min == v
       ensures exists v :: v in values[..] && max == v
{
  var i := values.Length - 1;
  var total := values[i];
  min := values[i];
  max := values[i];

  assert values[i..] == [values[i]];
  assert sum(values[i..]) == values[i];
  assert forall v :: v in values[i..] ==> min <= v;
  assert forall v :: v in values[i..] ==> max >= v;
  assert min in values[i..];
  assert max in values[i..];

  while 0 < i
    invariant 0 <= i < values.Length
    invariant total == sum(values[i..])
    invariant forall v :: v in values[i..] ==> min <= v
    invariant forall v :: v in values[i..] ==> max >= v
    invariant min in values[i..]
    invariant max in values[i..]
    decreases i
  {
    var oldI := i;
    var oldMin := min;
    var oldMax := max;

    i := i - 1;
    var x := values[i];

    assert oldI == i + 1;
    assert values[i..] == [x] + values[oldI..];
    assert values[i..][0] == x;
    assert values[i..][1..] == values[oldI..];

    total := x + total;
```

```
assert total == x + sum(values[oldI..]);
assert sum(values[i..]) == x + sum(values[i..][1..]);
assert sum(values[i..]) == total;

if x < oldMin {
  min := x;
  assert min in values[i..];
  forall v | v in values[i..]
    ensures min <= v
  {
    if v == x {
    } else {
      assert v in values[oldI..];
      assert oldMin <= v;
      assert min < oldMin;
    }
  }
} else {
  min := oldMin;
  assert min <= x;
  assert min in values[oldI..];
  assert min in values[i..];
  forall v | v in values[i..]
    ensures min <= v
  {
    if v == x {
    } else {
      assert v in values[oldI..];
      assert oldMin <= v;
    }
  }
}

if x > oldMax {
  max := x;
  assert max in values[i..];
  forall v | v in values[i..]
    ensures max >= v
  {
    if v == x {
    } else {
      assert v in values[oldI..];
      assert oldMax >= v;
      assert max > oldMax;
    }
  }
} else {
  max := oldMax;
  assert max >= x;
  assert max in values[oldI..];
  assert max in values[i..];
```

```
    forall v | v in values[i..]
      ensures max >= v
    {
      if v == x {
      } else {
        assert v in values[oldI..];
        assert oldMax >= v;
      }
    }
  }
 }

 assert i == 0;
 assert values[i..] == values[..];
 assert total == sum(values[..]);
 assert min in values[..];
 assert max in values[..];

 average := total / (values.Length as real);
 range := max - min;
}
```

Total Token Usage

Input tokens: 638
Output tokens: 4461
Reasoning tokens: 3584
Sum of 'total tokens': 5099

Experiment Timings

Overall Experiment started at 1780492061762, ended at 1780492149436, lasting 87674ms (87.67 seconds)
Iteration #1 started at 1780492061762, ended at 1780492149436, lasting 87674ms (87.67 seconds)